NPU COMPILER · V09 BACKEND · TVM 0.19 RELAX / TIR · 요약판

TVM 기반 LLM NPU Compiler

모델의 계산이 NPU 명령과 다음 token 이 되기까지 — 전체 pipeline 의 동작을 그림으로

계산의 의미

Relax · TIR · FP16 저장 /FP32 누산

배치

fusion · 64×64 tile 과 panel · SRAM/DRAM 주소

실행

ISA stream · linker · prefill/decode runtime

예제 : SiLU(X[64,128]·W^{T}+bias) · 두 matmul X·W1·W2 (SRAM 44 KiB) · Llama 3.2 3B (28 layers, prompt 7 tokens, capacity 9)
설정 : FP16 저장 / FP32 누산 · SRAM 8 MiB · tile 64×64 · 현재 탐색 개선 + 256-token E10 진행 상태 포함

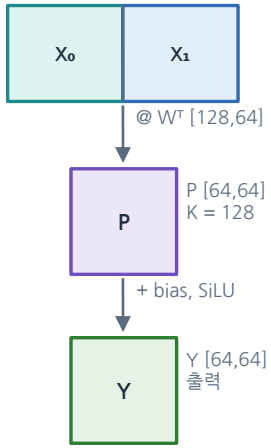
2026-09-29 탐색 개선 반영 · 근거 : d_compiler/docs (13 장 , examples, evidence)

한 장으로 보는 전체 pipeline: 같은 계산이 단계마다 다른 모습이 된다

작은 예제 SiLU(X[64,128]·Wᵀ+bias) 의 실제 산출물로 본 5 단계 . 후보 graph 마다 ③④⑤까지 내려가 (DMA bytes, words, SRAM peak) 를 얻고 그 비용으로 선택합니다 .

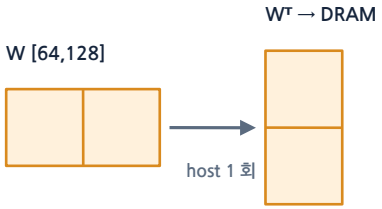
① 모델 → Relax

X [64,128] = 1×2 tiles




```
lv  = permute_dims(W)   # Wᵀ
lv1 = matmul(x, lv)
lv2 = add(lv1, bias)
gv  = silu(lv2)  # num_input=1
```

② Graph 정리 · LegalizeOps



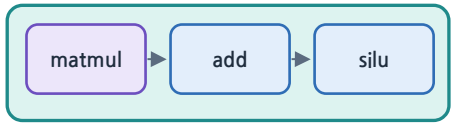
LiftTransformParams: weight 만 쓰는 계산을 분리 (LM head 751 MiB 도 1 회)

matmul pattern 4 → add pattern 0 → silu pattern 0

call_tir ×3 + PrimFunc (Relax 호출 순서는 남음)

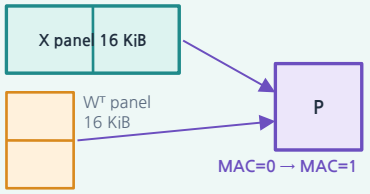
```
# matmul PrimFunc 요약
C32[i,j] = 0
C32[i,j] += f32(x[i,k])·f32(Wᵀ[k,j])
Y[i,j] = f16(C32[i,j])   # 한 번만
softmax: FP32 sum, RMSNorm: absmax
```

③ Fusion · Schedule



FuseOps 영역 : unfused 2,812 vs fused 1,577 words → 채택

SRAM (schedule 결과)



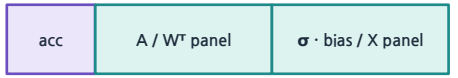
matmul: m/n panel, K chunk, loop 순서 , X/W cache 위치를 탐색 → 64×64 intrinsic 으로 tensorize. 후보마다 실제 명령을 내어 비용 측정 (중복 정규화 · 증명된 용량 초과 제외 · 시도 64 · legal 8 · 적응형 words 예산)

④ 메모리 계획

DRAM image 41,100 B (실제 주소)



SRAM: USMP offset 공유 (nibble)



peak 150,808 → 85,016 nib (bump vs greedy)

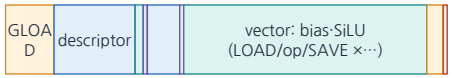
커널 사이 보관 (두 matmul, 44 KiB)



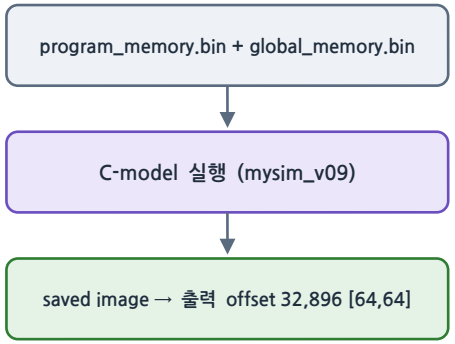
H 를 DRAM 에 쓰지 않고 다음 matmul 이 직접 읽음 → DMA 81,932 → 49,164 B, 미사용 DRAM pool 제거 (image 65,548 → 49,164 B)

⑤ ISA · 실행

명령 stream 1,577 words (정적 전개)



loop·branch 없음 · DMA packet 4 words



DMA 41,100 B · SRAM peak 42,508 B · LLVM reference(K=64 grouping) 와 최대 절대차 0 · fused/unfused bit-exact

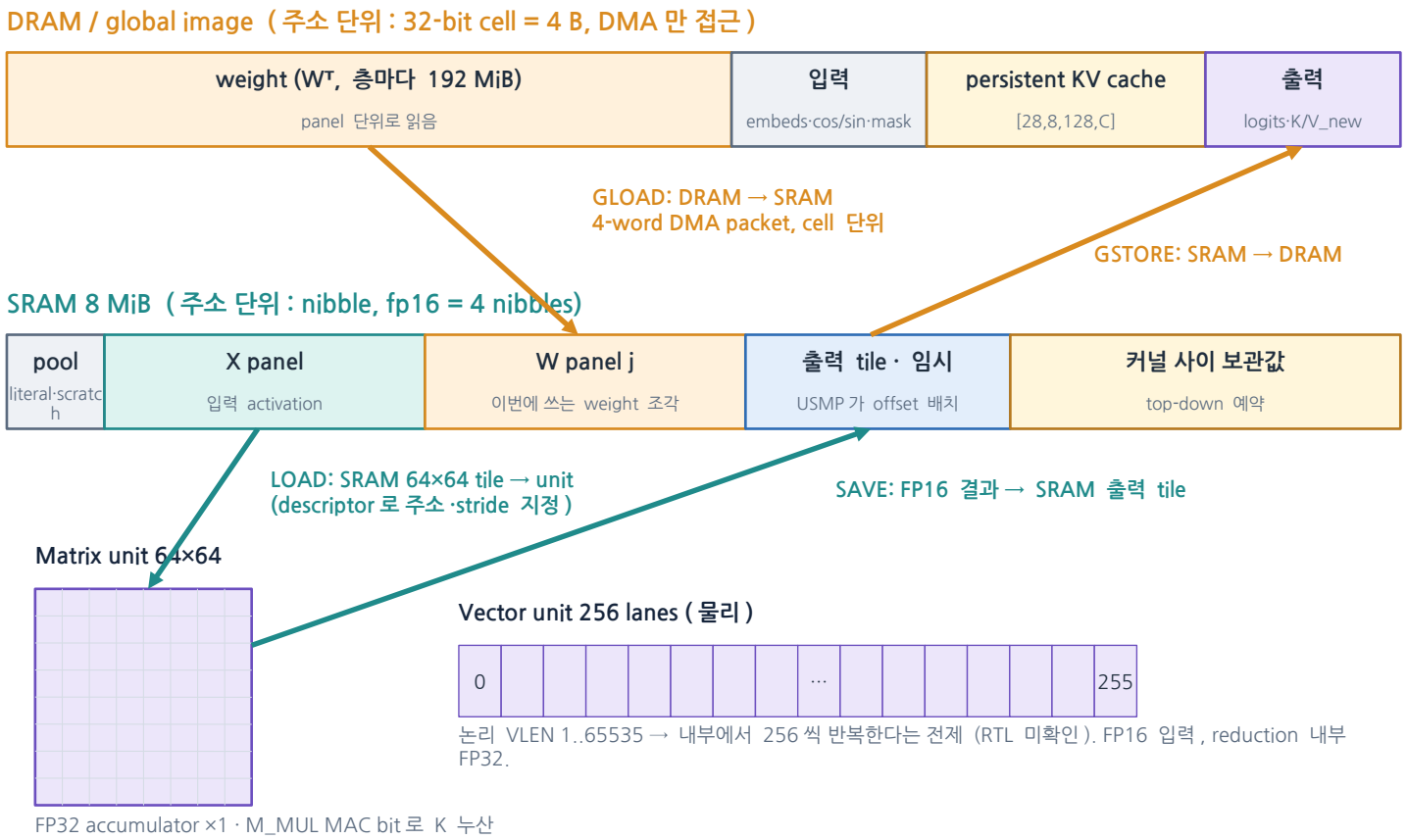
비용 (DMA bytes, words, SRAM peak) 으로 다음 후보 선택 — 기본 추가 탐색 120 s / None 해제 가능 · 요청 entry 가 비악화일 때 채택

" 모델을 compile 한다 " = graph 선택 + schedule + 주소 배치 + 명령 생성 . 실행은 그 뒤 별도 단계이며 , 입력만 바꿔 같은 명령을 재사용합니다 .

근거 : docs/examples/pipeline (01~09), bundle/program.json; npu_target.build → npu_build.lower_and_select → npu_link.compile_program

장치 : DRAM ↔ SRAM ↔ 연산장치 , 데이터는 panel 과 64×64 tile 로 움직인다

weight 는 DRAM 에서 panel 단위로 SRAM 에 올라오고 (GLOAD), 64×64 tile 이 연산장치로 들어가 (LOAD) FP32 accumulator 한 개에서 K 를 누산한 뒤 FP16 으로 SRAM 에 내려오며 (SAVE), 결과가 DRAM 으로 나갑니다 (GSTORE).



이동 명령	경계	DRAM 을 지나는가
GLOAD 0xA0	DRAM → SRAM	예 (4 words)
LOAD 0x90	SRAM → unit	아니오
SAVE 0x98	unit → SRAM	아니오
GSTORE 0xA8	SRAM → DRAM	예 (4 words)

주소 단위	값
plan.address (DRAM)	unit 2 B
DMA global 주소·stride	32-bit cell (×4 B)
SRAM descriptor 주소	nibble (÷2 B), 8-nibble 정렬
shape / VLEN	element

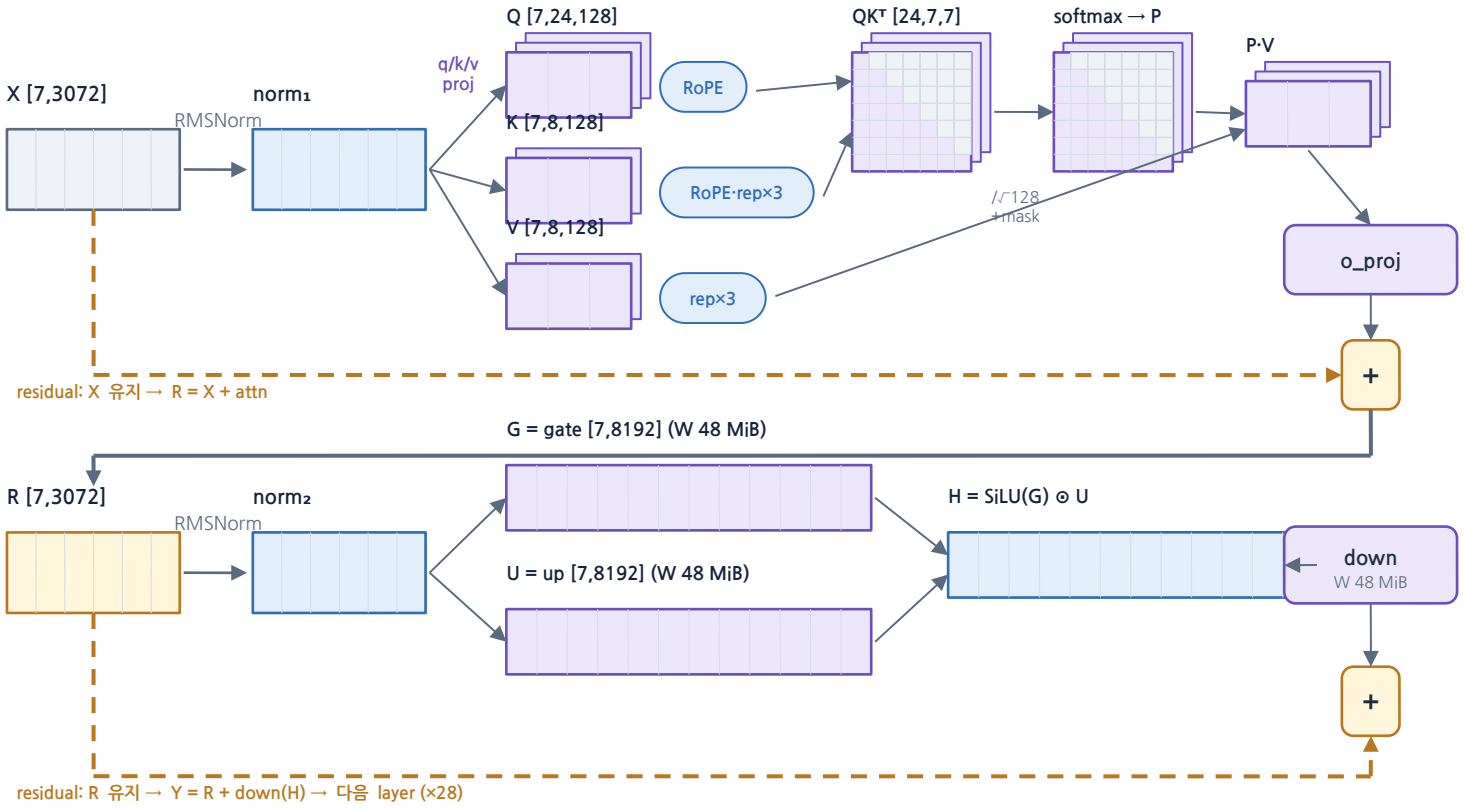
수치 계약 : FP16 저장 , K=64 단위 FP32 부분합을 accumulator 1 개에 누산 , partial spill 금지 . 자동 matmul 경로의 activation bit 는 OFF.
성능 지표는 DMA bytes → ISA words → SRAM peak (정적 , cycle 아님).

연산장치 안의 FP32 accumulator 는 SRAM buffer 가 아닙니다 . 어떤 projection weight 도 8 MiB 에 통째로 들어가지 않으므로 panel streaming 이 compiler 의 핵심 결정입니다 .

모델을 Relax 로 쓰고 , weight 만 쓰는 계산은 host 로 옮긴다

Llama 한 layer 를 tensor 모양으로 본 그래프 (왼쪽). 오른쪽 : LiftTransformParams 가 W transpose 를 host 함수로 분리해 W^T 가 DRAM image 의 정해진 주소에 한 번 놓이고 , device 함수는 token 마다 입력만 바꿉니다 .

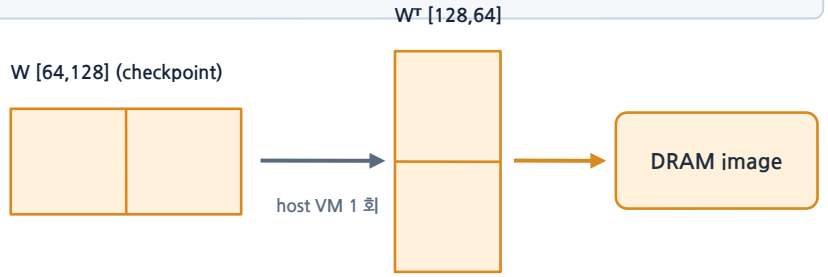
Llama 한 layer (prefill S=7): glyph = tensor, 겹침 = head 축 , 회색 삼각형 = causal mask



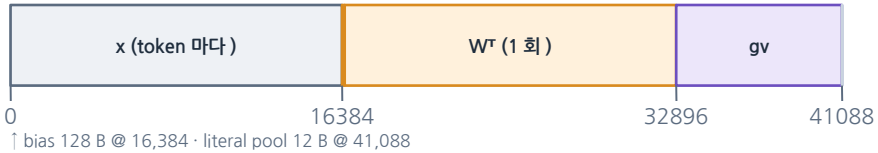
export 결과 : prefill(input_embeds [7,3072], cos/sin, mask [1,7,7], packed_params) → logits; decode(…, k_cache [28,8,128,9], v_cache, mask [1,1,10]) → (logits, K_new, V_new). num_input 이 실행 입력과 parameter 의 경계 .

LiftTransformParams (작은 예제의 실제 IR 요약)

```
# device: prefill(x, bias, lv)   # transpose 없음
lv1 = matmul(x, lv); lv2 = add(lv1, bias); gv = silu(lv2)
# host (1 회 ): prefill_transform_params(W, bias)
lv = permute_dims(W)  →  return (bias, lv)
```



DRAM image 41,100 B (bundle/program.json 실제 주소)



- LM head W^T 751.5 MiB 도 준비 단계 1 회 → token 마다 이동 0
- 반환 순서가 바뀌므로 runtime 은 signature(x, bias, lv) 로 binding

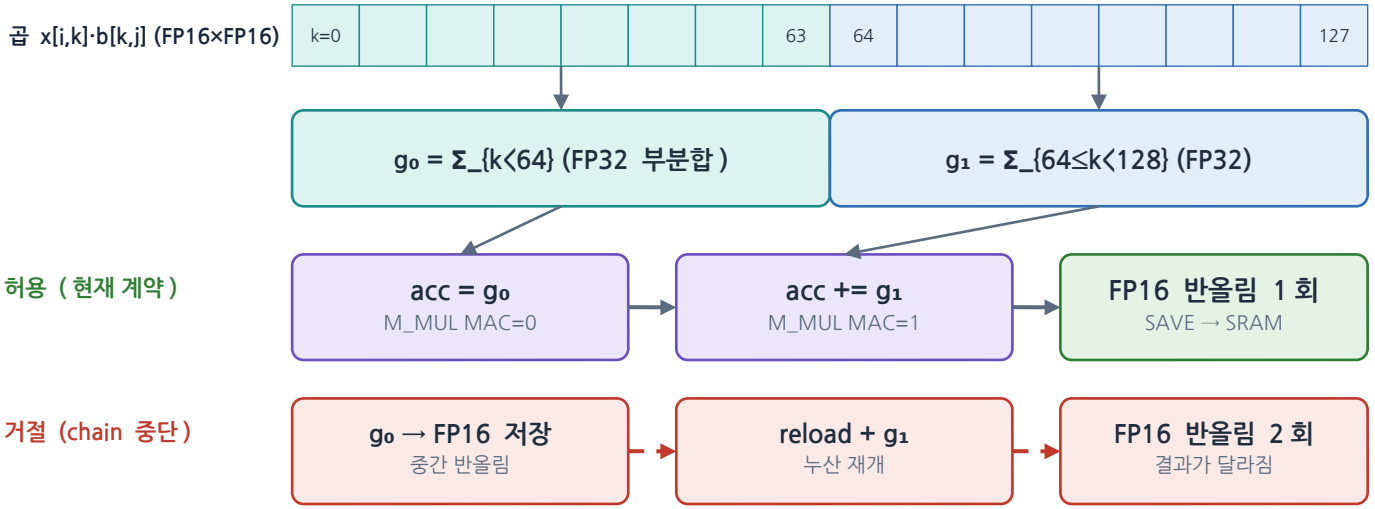
Compile 은 IR 을 분리하는 시점 , load 는 host 함수를 실제 weight 로 실행하는 시점 , run 은 입력만 바꿔 같은 프로그램을 재사용하는 시점입니다 .

근거 : nn_models/llama.py, docs/examples/llama_ir/01_authored_relax.py, pipeline/02_graph_prefix.py, bundle/program.json

② 계산의 의미

수치 계약 : FP16 저장 , FP32 누산 — K 전체 동안 accumulator 를 끊지 않는다

LegalizeOps 가 각 Relax 연산의 원소 계산을 TIR 로 정의합니다 (Relax 호출 순서는 남음). matmul 은 128 개 곱을 64 개씩 두 FP32 부분합으로 같은 accumulator 에 더하고 마지막에 한 번만 FP16 으로 반올림합니다 .



codegen 이 같은 출력 tile 의 K chain 사이에 다른 명령이 오면 "FP32 matrix chain interrupted" 로 거절 (accumulator 1 개 , partial spill 금지)

CPU reference 도 같은 grouping (split(k,64) + rfactor) — E1: LLVM 과 최대 절대차 0

```
matmul_rf[2,64,64]: g = Σ_{64 개} f32(x)·f32(b)      # 부분합 2 개
matmul[i,j] = g₀ + g₁                              # FP32 결합
compute[i,j] = f16(matmul[i,j])                    # 마지막 1 회 반올림
# FoldConstants 도 같은 기준 . 단순 flat-K loop 와 항상 bit-exact 는 아님
```

LegalizeOps 뒤 (Relax 는 남고 PrimFunc 가 생김)

```
lv1 = call_tir(matmul, (x, lv))   # pattern 4
lv2 = call_tir(add, (lv1, bias))  # pattern 0
gv  = call_tir(silu, (lv2,))      # pattern 0

# matmul PrimFunc: SSR 축 , T.init 0, FP32 buffer,
#    마지막 block 에서 f16 cast
```

NPU 정의 (custom legalize map)	왜 다른가
matmul	FP16×FP16 → FP32 sum → FP16 cast (K=64 grouping)
softmax	max → exp → FP32 sum → FP16 sum → divide. 마지막 축 1..65504 (exp ≤ 1)
rms_norm	a = max(absmax, √ε), u = x/a, z = u/√D, Σz² FP32 → FP16 (x² overflow 방지)
sum / mean / split	FP32 sum → FP16; split 은 view (zero-copy)
그 외	TVM 기본 legalization (add, silu, permute_dims, concat, …)

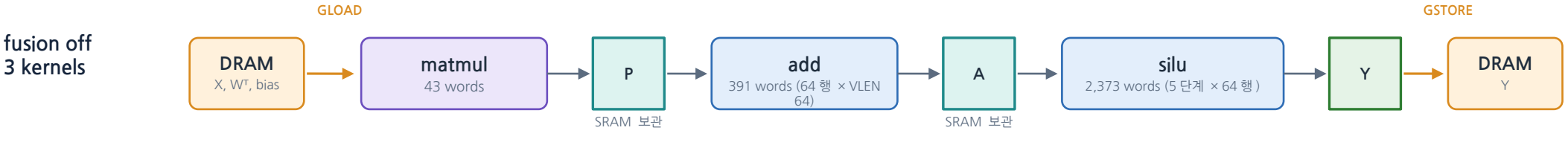
지원 밖 (길이 70,000 softmax, FP32 RMSNorm 등) 은 ValueError 로 거절 . 조용히 근사하지 않음 .

Fusion·schedule 은 저장 위치를 바꿀 뿐 이 수치 계약을 바꾸지 않습니다 . 정밀도 정책은 고정이며 자동 오차 예산 탐색은 없습니다 .

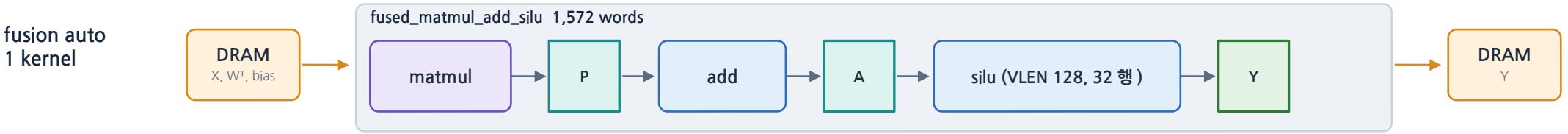
근거 : npu_compiler/npu_legalize.py, npu_numeric.py, docs/examples/pipeline/03_unfused_semantic_tir.py, 08_llvm_reference.py

Fusion 은 커널 수가 아니라 실제 프로그램 비용으로 고른다

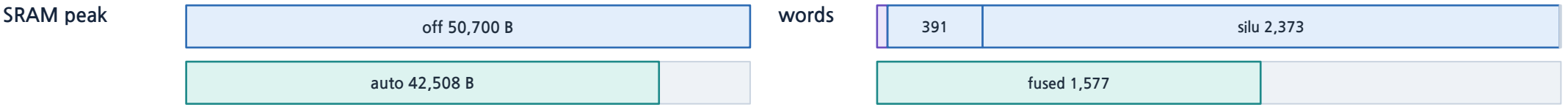
작은 예제의 분리 (off) vs 결합 (auto): 두 경우 모두 중간 tile P·A 가 SRAM 에 머물러 DMA 는 같고 (41,100 B), 커널 경계의 descriptor·LOAD/SAVE 반복과 짧은 vector 행이 words 와 SRAM 작업 공간을 바꿉니다 .



커널 경계마다 : 새 descriptor 설정 + P/A 를 SRAM 에서 다시 LOAD/SAVE (DRAM 왕복은 없음)



한 커널 안에서 이어짐 : descriptor 반복 감소 , 축 결합으로 vector 행 64 → 128 (명령 수 절반), USMP offset 공유



선택 규칙 : cost = (DMA bytes, words, SRAM peak) 사전식 · 영역 검사 (npu_fusion.select) → 전체 module 비교 (lower_and_select)

요청한 entry 가 모두 비악화 + 하나 이상 개선일 때 채택 . 기본 시간 120 초 / E10 은 prefill 만 , 시간 제한 None.

27 조합 → 같은 IR 인 source 제거 (회색 18, E10 prefill) → 9 구성 · ✓ 과거 7-token 기록



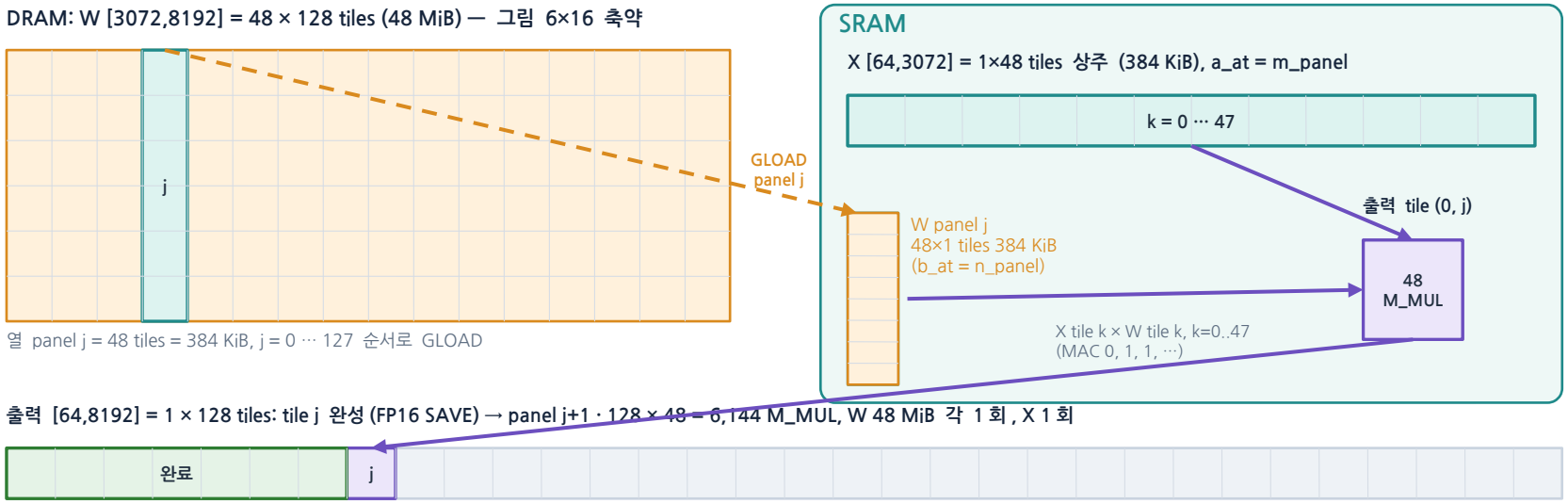
✓ 후보 0 (모두 off) 선택 · 후보 1 에서 120 s 소진 (7-token, 189.7 s) · ≡ E10 prefill 에서 같은 source

E1: 전송량은 같아도 명령과 작업 공간이 줄어든 사례 (2,812 → 1,577 words, 50,700 → 42,508 B). 이득의 형태는 예제마다 달라 실제 비용으로 확인합니다 .

근거 : npu_compiler/npu_fusion.py, npu_build.py, docs/examples/pipeline/stage_summary.json (E1), evidence/current/llama_selection.json

Matmul: 64×64 tile 은 고정 , 자유도는 panel(무엇을 SRAM 에 얼마나 오래) 과 순서

실제 폭 gate projection X[64,3072]@W[3072,8192]: W 는 48 MiB 라 X(1×48 tiles, 384 KiB) 를 상주시키고 W 열 panel j(48×1 tiles) 를 128 번 교체합니다 . 아래는 두 matmul 예제 두 번째 커널의 실제 20 words.



탐색 축 : m/n panel, K chunk, outer/inner 순서 , X/W cache 위치 (a_at/b_at). K 는 항상 최내측 (accumulator 1 개)

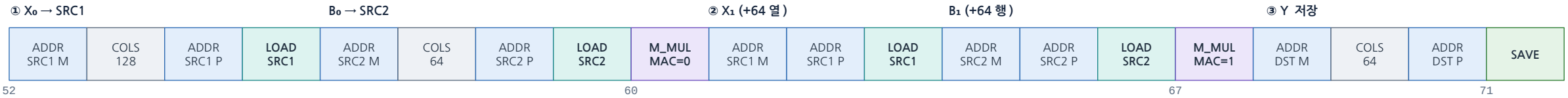
검사 : 같은 순회 · 복사 주기의 표기는 후보 하나로 정규화 , 증명된 SRAM 초과는 제외 → 추정 DMA 순으로 실제 schedule·compaction·USMP· 명령 생성 → (DMA, words, SRAM). 시도 64, legal 8, words max(100k, 기준 ×8)

선택형 : MetaSchedule ReplayFunc 추첨 탐색 · trace DB 재사용 (기본 off, 마지막 슬라이드)

선택 기록 : gate/up ("A",64) X 유지 · down ("B",256) W 4 MiB 유지 , H 는 SRAM 보관값

tensorize: 64×64 조각 → npu_gemm_acc intrinsic → codegen 이 descriptor·LOAD·M_MUL 로

실제 명령 : H[64,128] @ W2[128,64] (word 52-71, 20 words) — ① X₀·B₀ MAC=0 ② X₁·B₁ MAC=1 ③ SAVE 1 회



descriptor(ADDR/COLS) 를 바꾸고 LOAD → M_MUL(MAC bit) → 마지막에 SAVE. 두 번째 tile 에서 COLS 가 없는 이유 : rows/cols 가 그대로라 peephole 이 생략 . loop·branch 명령은 없음 (정적 전개).

K chunk 숫자만 보고 DMA 를 판단하면 안 됩니다 . cache stage 가 어느 loop 에 있는지 (a_at/b_at) 와 순회 순서가 재읽기 횟수 , 곧 DMA 를 정합니다 .

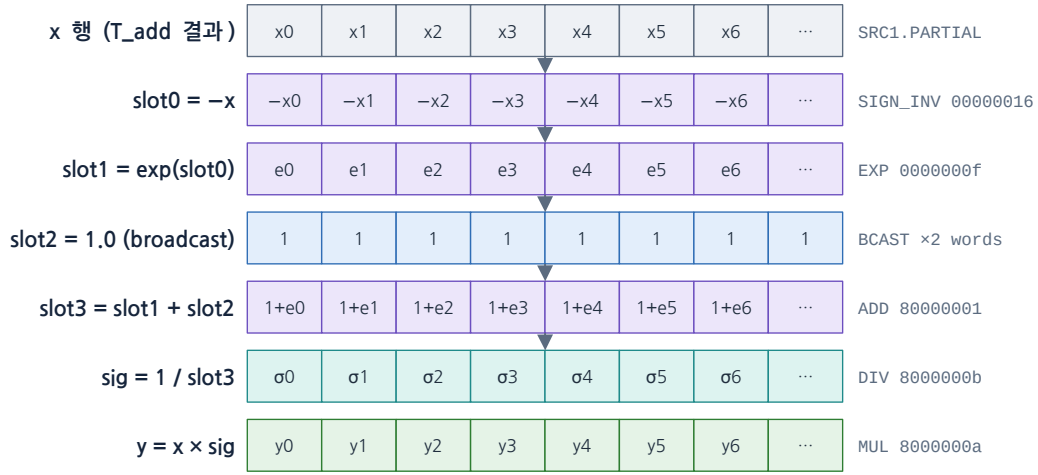
근거 : npu_schedule_space.py, npu_schedule.py, npu_intrin.py, evidence/current/refinement.json, two_matmuls_recorded/program.json (words 52-71)

③ 배치

Matmul 이외 : SiLU 하나가 vector 명령 5 개 , attention 은 query block 으로 score 를 SRAM 에서 소비

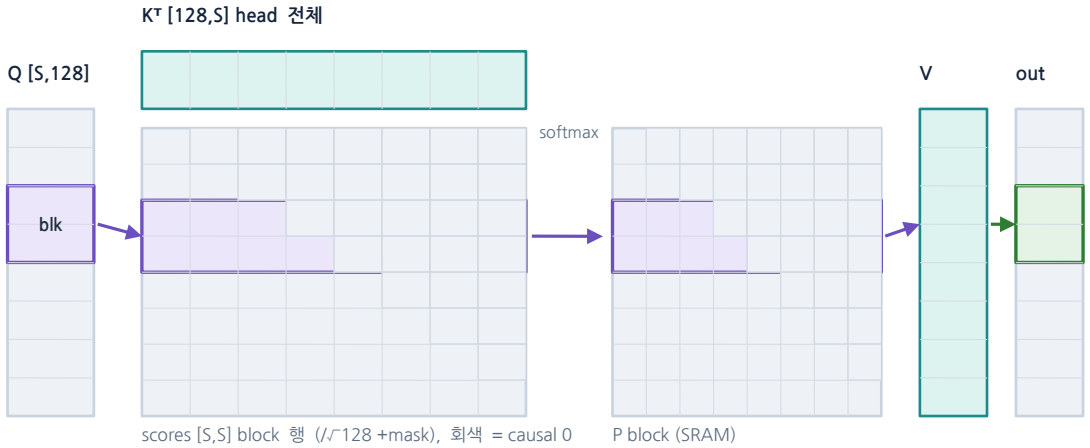
원소별 연산은 VLEN 길이의 행이 SRAM scratch slot 을 거치며 단계마다 descriptor·LOAD·SAVE 가 붙습니다 (왼쪽 , 실제 opcode). Full-attention fusion 은 query 행 block 마다 K/V 전체와 score 행 전체를 SRAM 에 두고 정확한 softmax 를 합니다 (오른쪽).

SiLU 한 행 (VLEN 128, FP16 scratch) — 오른쪽은 실제 opcode word



단계마다 VLEN·ADDR·LOAD(·LOAD)·OP·ADDR·SAVE → E1 fused 1,577 words 중 ADDR(0x80) 606, LOAD/SAVE 388/258. 행이 안 들어가면 stripe 반분 , 연속 축 결합으로 64 행 ×64 → 32 행 ×128 (명령 절반).

Full-attention fusion: query block (진한 행) 만 계산 , score 행 전체를 SRAM 에서 softmax



schedule: row block 을 head loop 바깥에 (reorder) → block 마다 모든 head. scores·max·exp·sum·PV acc 는 SRAM 에만

block 크기 : 가장 큰 block 부터 , SRAM 초과 시 반분 (floor = min(rows,64)). K/V 는 block 마다 재읽기

S=3072: 24-head score 432 MiB → head 1 개 64 행 block 384 KiB. 조건 : rank-3 FP16, m·d·t 64 배수 (decode m=1 은 제외)

없는 것 : key 방향 online softmax (FlashAttention). 한 block 이 행 전체를 가져 softmax 는 정확

Softmax / RMSNorm 도 같은 vector 경로 : REDUCE_MAX → SUB → EXP → REDUCE_SUM(FP32 내부) → DIV; RMSNorm 은 absmax scale 후 Σz². 물리 256 lanes vs 논리 VLEN ≤ 65535 는 RTL 미확인 항목이며 현재 비용 모델에 lane 활용률은 없습니다 .

words 는 짧은 vector 행과 descriptor 반복에 좌우되고 , attention 은 384 KiB score 만이 아니라 K/V·mask· 확률 · 출력이 함께 SRAM 에 있어야 합니다 .

근거 : tir_codegen_v09.py (_materialize, _sigmoid), docs/examples/pipeline/09_packets.json, npu_intrin.schedule_attention_blocked, npu_schedule._fit_attention

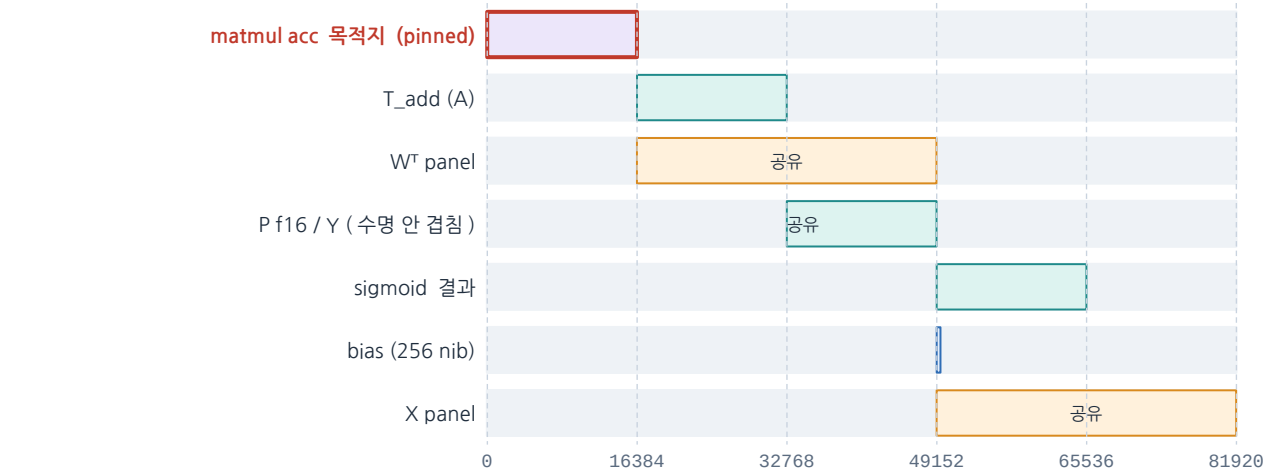
메모리 계획 : DRAM pool, 커널 안의 offset 공유, 커널 사이 SRAM 보관

① TVM StaticPlanBlockMemory 가 수명이 겹치지 않는 tensor 를 같은 pool 에 두고 backend 가 flat 주소를 줍니다 . ② USMP 가 커널 buffer 의 충돌을 분석해 offset 을 공유합니다 . ③ 보관 탐색이 중간값 H 를 SRAM 위쪽에 예약하고 남은 용량으로 커널을 다시 schedule 합니다 .

① DRAM image (flat, unit 2 B): parameter → 상수 → storage pool(수명 안 겹치는 tensor 공유) → 출력 → literal



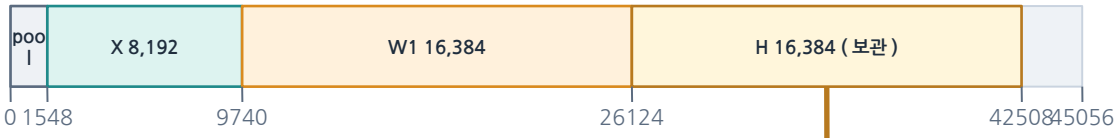
② USMP (작은 예제 fused 커널): 충돌 없는 buffer 가 offset 공유 — nibble



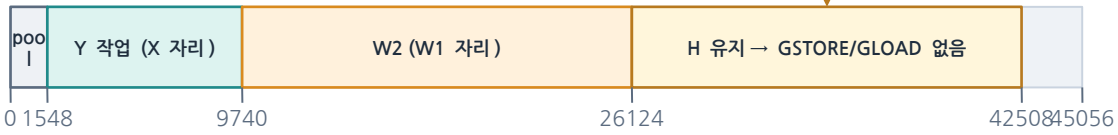
peak 150,808(bump) → 85,016 nibbles (32,896 B 절약). gemm 목적지는 SAVE 가 지연되므로 pinned(모든 buffer 와 충돌) 로 offset 0 을 혼자 씀 . greedy_by_size / greedy_by_conflicts vs bump 중 최소 peak 채택 .

③ 커널 사이 보관 (두 matmul, SRAM 44 KiB, 실제 주소 bytes)

커널 1: X @ W1 → H (H 를 SRAM 에 직접 SAVE)



커널 2: H @ W2 → Y (H 를 그 자리에서 직접 LOAD)



E2 (실제 실행)	DMA bytes	words	SRAM peak	DRAM image
32 KiB (보관 불가)	81,932	88	26,124	65,548
44 KiB 보관 off	81,932	87	34,316	65,548
44 KiB 보관 on	49,164	77	42,508	49,164

H 의 저장 · 재읽기 32,768 B 제거 , H 의 DRAM pool 제거 (주소 재배치). 보관은 tensor 전체만 , beam 4 + all-DRAM 비교 , 이득이 실제 비용일 때만 채택 . KV cache· 출력 ·parameter 는 ABI 계약으로 보호 (이름이 아님).

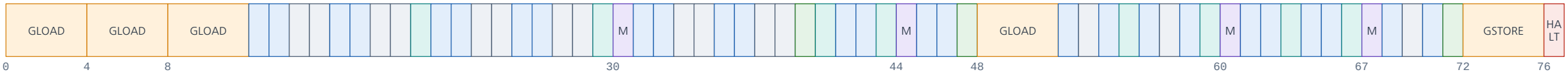
커널을 합치지 않아도 SRAM 만으로 연결할 수 있습니다 . 같은 주소가 나중에 다른 값이 될 수 있으므로 보관 탐색은 " 값의 버전 " 을 키로 삼습니다 .

근거 : npu_memplan.py, npu_sram.py, npu_memory_search.py, bundle/program.json sram_layouts, two_matmuls_recorded/program.json, examples/memory/measurements.json (E2)

ISA: 정적으로 전개된 명령 stream — descriptor 설정 , LOAD/SAVE, M_MUL, DMA packet

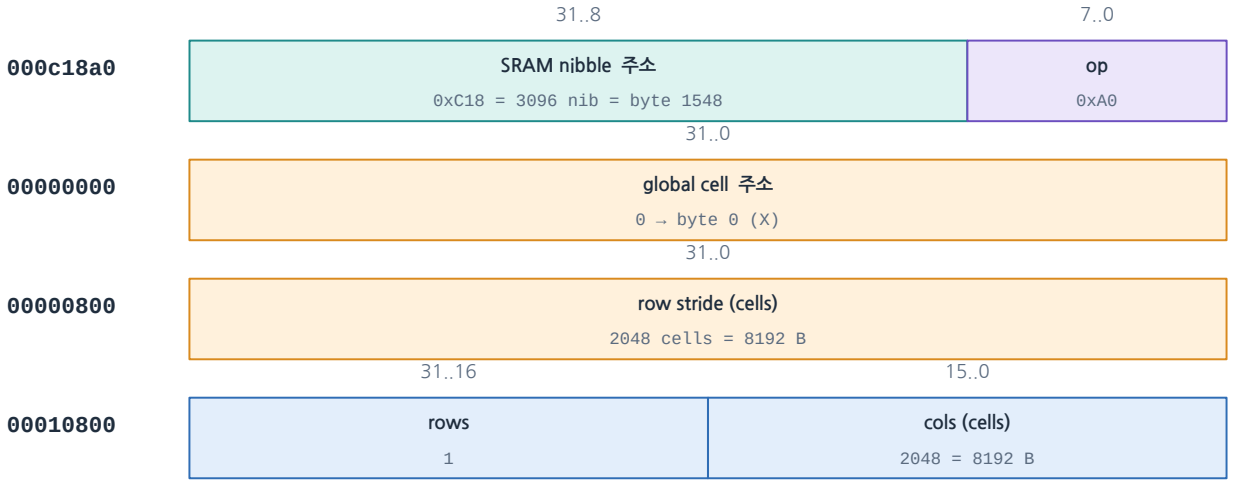
두 matmul 예제의 77 words 전체 (위) 와 실제 word 해독 (아래). walker 가 TIR loop 를 완전히 unroll 하므로 loop·branch 명령이 없고 , linker 가 커널 stream 을 이어 붙인 뒤 peephole·forwarding·DRAM 압축으로 마무리합니다 .

77 words 전체 = 62 instructions (DMA 5 개 × 4 words + 1-word 명령 57 개): 주황 DMA · 파랑 ADDR · 회색 ROWS/COLS · 청록 LOAD · 초록 SAVE · 보라 M_MUL · 빨강 HALT



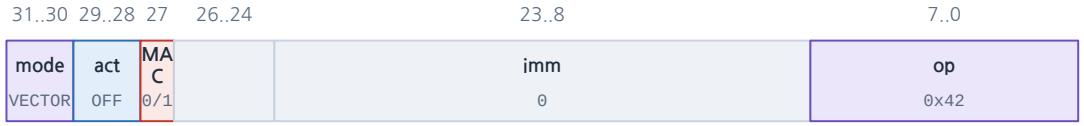
0-3 상수 pool · 4-47 커널 0 (W1·X GLOAD, tile 0 MAC=0 SAVE, tile 1 MAC=0 SAVE → H 직접) · 48-75 커널 1 (W2 GLOAD, MAC 0/1, SAVE Y, GSTORE) · 76 HALT

DMA packet 해독 (word 8-11: X 를 SRAM byte 1548 로)



단위 : SRAM nibble(fp16 = 4), global 32-bit cell(×4 B), plan unit 2 B. 한 숫자를 다른 단위로 읽으면 2 배 /4 배 어긋남 . DMA 는 dtype 없는 cell 이동 .

M_MUL word: 80000042 (MAC=0) vs 88000042 (MAC=1)



Descriptor = 상태 register	뜻
0x80 ADDR	SRC1/SRC2/DST × MAIN/PARTIAL 주소 (16-bit 절반씩)
0x88/0x89 ROWS/COLS	rows/cols + dtype. MAIN cols = row stride (128 el = 256 B)
0x90 LOAD / 0x98 SAVE	PARTIAL 64×64 view 를 MAIN stride 로 읽고 / 쓰기
peephole	값이 같을 때만 중복 설정 삭제 (커널 1 에 0x88 없음)

Linker: 상수 pool GLOAD → 커널 stream 연결 (binding 순서) → HALT → residency forwarding → peephole → joint vs all-DRAM 비교 → 미사용 DRAM pool 제거 → KV append 재구성 . Python/native walker 는 같은 contract hash 를 공유하고 C-model counters 와 대조합니다 .

words 는 " 명령 저장 공간 " 이며 cycle 이 아닙니다 . bundle(program_memory.bin + program.json + IR + image) 만으로 실장치 latency 를 주장하지 않습니다 .

근거 : two_matmuls_recorded/program_memory.bin, program.json; tir_codegen_v09.py, v09_walker.cc, npu_link.py, peephole.py, isa_v09.py

⑤ 실행

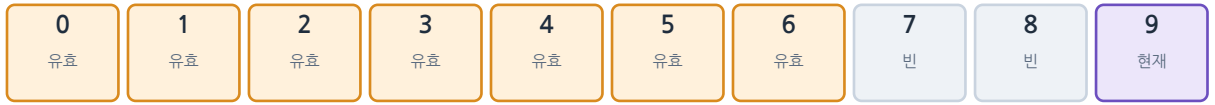
Runtime: prefill 1 개 + decode 1 개 프로그램 , cache 는 DRAM 에 남고 새 K/V 는 DMA tail 이 슬롯에 쓴다

Decode 는 capacity C 개 슬롯 + 현재 token 으로 항상 C+1 길이를 attention 하고 mask 가 유효 길이를 전달합니다 . host 는 embedding·cos/sin·mask 만 patch(6,676 B) 하고 , program 끝의 append tail 이 새 K/V 를 슬롯 L 에 옮깁니다 .

prefill(7 tokens) → decode → decode: 실제 3-token 기록

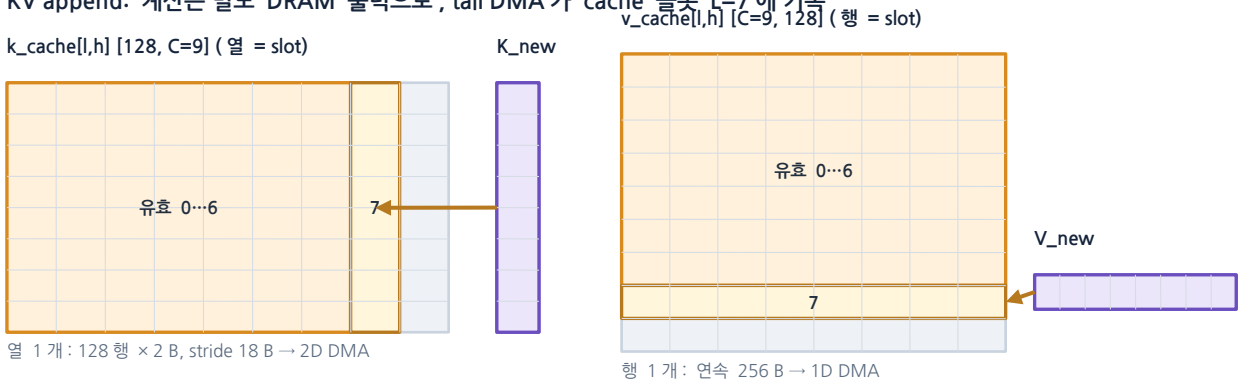


decode @7 이 보는 논리적 입력 : capacity C=9 + 현재 = 10 슬롯 , 유효 길이 L=7



`mask [1,1,10]: 0 0 0 0 0 0 0 −65504 −65504 0` → softmax 가 빈 슬롯의 기여를 0 으로 (계산 자체는 수행)

KV append: 계산은 별도 DRAM 출력으로 , tail DMA 가 cache 슬롯 L=7 에 기록



열 1 개 : 128 행 × 2 B, stride 18 B → 2D DMA

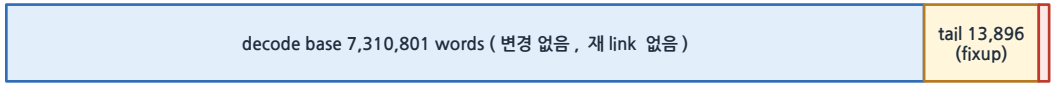
행 1 개 : 연속 256 B → 1D DMA

step 마다 실제로 바뀌는 바이트 (DecodeSession · PersistentTransport)

global_memory.bin ≈ 6.43 GB



program_memory.bin 7,324,697 words



대상	step 사이
decode 계산 명령 · DRAM weight	재사용 (host rewrite 0 B)
embedding · cos/sin · mask	host patch 6,676 B (plan.address 위치)
KV append 명령 주소	position fixup (suffix patch 55,568 B)
persistent DRAM cache	saved image → 다음 입력 (rename)
C-model	호출마다 전체 image read/write (기능 검증용 , cycle 없음)

실행	token	HF 일치	logits cosine	최대 절대차
prefill	358	예	0.9999977	0.035
decode @7	2846	예	0.9999958	0.023
decode @8	4560	예	0.9999930	0.039

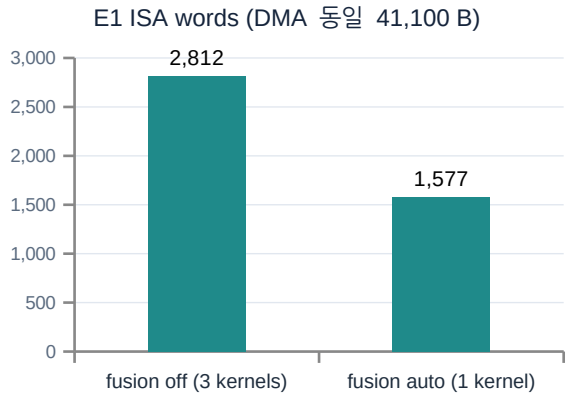
prefill 8.58M words / DMA 6.46 GB, decode 7.31M words. compiler 통계 ·host patch·simulator I/O 는 서로 다른 측정 층 .

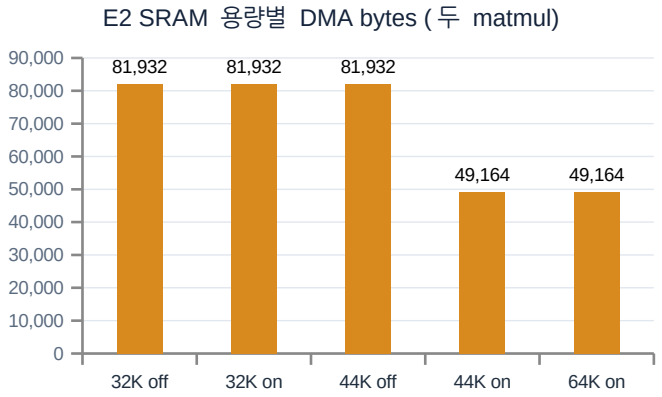
Greedy argmax 는 host 에서 . 마지막 예측 token 은 아직 NPU 에 넣지 않았으므로 생성 token 수와 decode 실행 횟수가 한 번 차이 납니다 .

근거 : npu_generate.py (generate, DecodeSession, length_mask), npu_cache.AppendPlan, v09_runtime.PersistentTransport, evidence/current/llama_validation.json

검증된 것과 아직 없는 것

작은 예제는 실제 C-model 실행에서 fused/unfused bit-exact, LLVM reference 와 최대 절대차 0. 전체 Llama 3.2 3B 는 HF 와 같은 greedy token 3 개 , logits cosine ≥ 0.99999. 아래 수치는 모두 보존 기록이며 ISA words 나 DMA 를 실장치 latency 로 바꾸어 말하지 않습니다 .





실험	결과	검증 수준
E3 직접 SRAM 출력 ·DRAM 압축	words 85 → 77, image 65,548 → 49,164 B, bit-exact	보존 실행
E4 실제 폭 MLP (M=64)	5 kernels, 140,594 words, DMA 152 MB, SRAM peak 7.45 MB	정적 compile
E5 compile 속도	M=7: 84.2 → 22.1 s, trial 347 → 112, binary hash 동일	단일 관측
E6 전체 Llama 3B	358 / 2846 / 4560 = " I'm trying", cosine ≥ 0.99999	보존 실행 (948 s, cold compile 아님)
E7 회귀 gate	17 suites exit 0, skip 없음	보존 실행

현재 지원 경계

Shape·control flow: 정적 shape, 지원된 정적 호출의 직선 전개 (dynamic control flow 거절)

연산 : softmax 마지막 축 1..65504, RMSNorm FP16, full-attention 은 rank-3 FP16 64 배수 (decode 제외), RoPE default/linear/llama3

메모리 : tensor 전체 보관만 , 고정 호출 순서 , 완전 미사용 pool 만 제거

Runtime: C-model adapter (image/file patch). RTL· 실장치 runtime 검증 아님

양자화 : W8A16/W8A8 kernel bit equality 까지 . generation 에는 미연결

남은 검토 (착수 목록 아님)

- Vector 256 lanes vs 긴 VLEN 의 RTL 계약 · vector 비용 모델 (lane 활용률 ·cycle)
- Attention key-online softmax · decode capacity 낭비 (bucket, 최종 슬롯 직접 저장)
- 부분 tile 보관 · 퇴거 · 호출 순서 변경 · 사용 중 pool 내부 압축
- 성능 모델 : RTL/ 장치 timing 확보 뒤 DMA→words→SRAM 순서와의 상관 확인

지표 (DMA/words/SRAM) 로 확인되지 않는 개선은 채택하지 않습니다 . 수치 정확성 (HF 대비) 과 프로그램 비용은 별개의 표이며 같은 source hash 를 가리킵니다 .

근거 : docs/current/11_experiments.md (E1~E7), 12_operations_limits.md, evidence/current/*

09-29 탐색 개선 : 중복 후보 정규화 , 증명된 용량 초과 제외 , 적응형 words 예산 — MetaSchedule 은 선택형

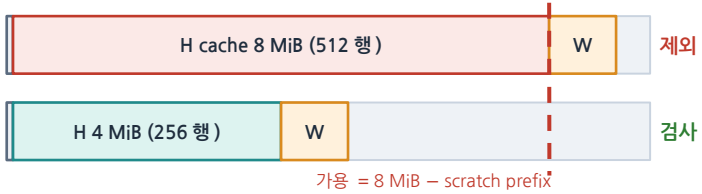
같은 tile 순회 · 복사 주기의 표기는 한 후보로 합치고 (512-token down 22,236 → 5,034), 두 입력 cache 합이 8 MiB 를 넘는 1,596 개를 증명으로 제외한 뒤 3,438 개를 검사 목록에 둡니다 . 기본 탐색은 space, ReplayFunc 추첨과 trace DB 는 옵션입니다 .

① 표기 중복 → signature 하나 (toy [128,256]×K128)



loop 위치 조합 1,728 → 후보 73 (현재 코드 계산). 복사 위치 (prefix) 가 다르면 , 예 : b_at=n_panel 의 tile 마다 vs 2 tile 마다 , 별도 후보로 유지 .

② 두 입력 cache 합으로 증명된 초과만 제외



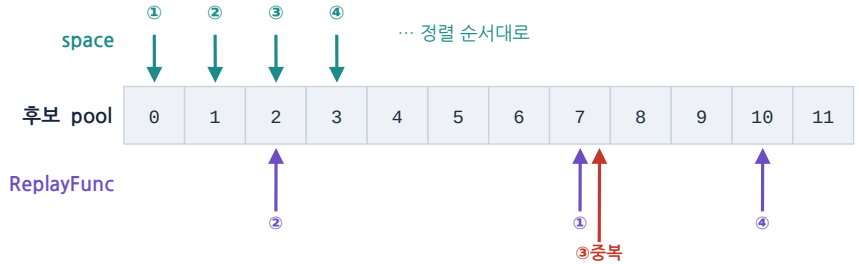
조건 : 정렬 rank-2 FP16 직접 입력 2 개 , 64 배수 (ordinary). padding·computed producer 는 순서만 뒤로 . 512-token down: 5,034 중 1,596 제외 → 3,438 검사 목록 (E9). gate M=64: 203 → 162 (계산).

③ 예산과 종료 사유 (stop_reason)

legal_budget	word_budget	attempt_budget	exhausted /
legal ≥ 8	legal ≥ 2 ∧ Σwords ≥ max(100k, 기준 ×8)	검사 ≥ 64 (실패 포함)	database_hit

incumbent(legacy A/B/B_stream/None, legal 2· 시도 64) 를 먼저 잡고 확장 후보를 검사 . 기준 words 가 큰 커널일수록 words 한도가 같이 커져 큰 커널에서 너무 일찍 끝나지 않음 (E8 512-down: 16 회 모두 용량 초과 → 개선 후 4 회 검사 = 합법 4 개). E10 실행기는 legal 16 · 시도 128 · words 10^9.

④ 검사 순서 : space(추정 DMA 순) vs ReplayFunc(sample_categorical 추첨) · 선택된 trace 는 JSONDatabase 에 저장 · 재생 — 평가는 공통



중복 추첨은 schedule 을 다시 만들지 않고 시도만 소비 . 두 순서 모두 schedule → lowering → SRAM 배치 → 명령 생성 → (DMA, words, SRAM). Llama 4 개 커널 : 최종 binary 동일 (E8) · 512-down 개선 후 4 회 /4 legal, 이전과 같은 명령 (E9).

```
# JSONDatabase ( 선택형 , 기본 off)
Workload: PrimFunc + 실행 조건
  (shape·dtype·scope·SRAM·scratch
   · 수치 · 버전 ) − 탐색 설정
TuningRecord: trace, run_secs=None
reuse: 재생 · 재검증 → 탐색 0 회
tune: incumbent → 개선 시만 기록
# 작은 예제 : 새 후보 TIR 8→0, bit-exact
```

⑤ E10 중단 : 28-layer · 256-token prefill

조건	값
graph 시간	None (120 s 제한 해제)
graph 구성	27 → source 3 × generic 3 = 9
matmul 예산	legal 16 · 시도 128 · words 10^9
메모리	joint on · beam 4 · DB 미사용
상태	메모리 증가로 중단 · 전체 재검증 미실행

비용 ·fusion 채택 · 출력 일치는 완료 전까지 주장하지 않음

개선된 것은 탐색의 낭비와 기록 재사용입니다 . 512-token down 의 최종 명령은 개선 전과 같았고 (E9), 전체 모델 결과 (E10) 는 완료 전까지 주장하지 않습니다 .

근거 : npu_schedule_space.py, npu_schedule.py, npu_meta_schedule.py; evidence/current/llama_schedule_refinement.json (E9), llama_prefill256_full_search_progress.json (E10)